Архитектура защищённой платформы корпоративных знаний

Действующая реализация после 20.10.2026

Следующие уточнения имеют приоритет над прежними описаниями ниже. Final authorization - BEGIN IMMEDIATE release snapshot: evidence/graph/provenance/session/identity/history/audit согласуются перед commit. SSE отправляет один проверенный результат. Достоверность права относится к commit; последующие отзывы не отзывают переданные байты. Версии/текст источников проверяются повторно, history не хранит цитат.

TokenBudget использует реальный vLLM tokenizer/chat template с JSON/system/question/полными source units, reserve700 + margin64 при max4096. Неумещающийся context даёт extractive-context-limited; invalid model citation заменяется полными units с явным mode и повторной валидацией. Graph traversal идёт через authorized mentions, включая singleton chunks; bounded ≤2 hops/32 chunks/128 entities → rerank6. Grounding сохраняет целый unit и offsets; semantic truth/OCR accuracy не гарантированы.

Derived stores теперь immutable versioned с единым SQL active pointer, validate перед publication, retain/rollback; destructive delete active коллекции отсутствует. Readiness отделён от liveness, admission bounded, HTTP accounting охватывает auth/validation failures. Реализованы проверяемые backup/restore, audit chain, retention dry run, quality/release gate и явная FinOps estimate. Industrial SSO/Vault/HA/mTLS остаются prerequisites. ADR-007/008 и remediation/RUNBOOK описывают границы и trade-offs, Model Card - governance.

Назначение и границы

Платформа помогает инженеру найти подтверждённые сведения в технических документах и связать оборудование через общий контекст. Пример: насос Н-17 упомянут с контуром К-2 в одном документе, а контур К-2 - с датчиком Д-8 в другом. GraphRAG находит двухшаговую связь и возвращает оба источника. Отдельный документ с секретным бюджетом Н-17 не доступен роли staff. Платформа не управляет оборудованием, не утверждает истинность произвольных co-mention связей и не заменяет технического специалиста.

Основание: задание «Проект», лекции C4/HLD/LLD, RAG (страницы 41 и 47-49), AI-агенты (Supervisor, Plan-and-Execute), ADR, Security by Design, наблюдаемость. Продуктовая гипотеза - сократить время поиска регламентов без нарушения доступа. Реальные документы/метрики предприятия не предоставлены, поэтому тестовые документы явно синтетические.

Архитектурные драйверы

Все inference-вызовы выполняются внутри разрешённого контура. Runtime не требует Интернета, pretrained artifacts доставляются до air-gap. ACL применяются к чанкам и provenance edges до retrieval/traversal, повторно перед выдачей. Сессия принадлежит конкретному субъекту и tenant. Секреты не попадают в LLM state/контекст/telemetry. Запросы ограничены 2000 символами, документы 25 MiB/100 страницами, graph hops ≤2, retrieval context ≤6 chunks, agent recursion ≤20, planner attempts ≤2, inference timeout 30 с. Эти значения задают границы MVP, не доказанные SLO банковской эксплуатации.

Предлагаемый целевой SLO: p95 end-to-end ≤5 с на 4 параллельных запросах с 4096-token context cap, availability 99,5% для пилота. Задание не задаёт эти числа. Фактические замеры двух GPU kernels и CPU приведены в нагрузочном отчёте; успешный короткий прогон не доказывает availability, качество на Golden Set или запас под другой workload.

Представления C4 и специализированные схемы

архитектура.drawio содержит отдельные страницы C1 Context, C2 Container, C3 Agent, C4 Code, Deployment, Sequence, ER и Data Flow. Статическая версия - архитектура.pdf. Четвёртый уровень C4 представлен явно: диаграмма кода Store/Agent/Generator/State/Principal. Deployment не подменяет code diagram. Уровни абстракции не смешаны.

C1: сотрудник и администратор взаимодействуют с платформой; ERP/CRM/файловый архив - внешние источники. Коннекторы ERP/CRM являются проектными интерфейсами, в реализованном MVP загрузка идёт через CLI файлов. C2: Control Plane - frontend/API gateway/Agent Orchestrator и ingestion administration; Data Plane - SQL metadata, Vector DB, provenance graph, OCR/serving. API и agents собраны в один backend образ MVP, роли компонентов разделены в коде; дробление на множество сервисов не требуется для проверки гипотезы.

C3: input guard, scoped memory, planner, tools interface, retriever, graph tool, reranker, generator, output validator. C4 отражает реальные методы, а не обещанные классы. Локальное состояние LangGraph сохраняет steps/chunks/edges/evidence. SQLite history хранит только вопрос после PII masking и source IDs, а не ответ, который можно повторить после отзыва права. При использовании memory источники заново авторизуются.

End-to-end Data Flow

Администратор определяет tenant и роли для документа. Text/Markdown читаются как UTF-8. PDF парсится с page provenance в layout mode; страницы без текстового слоя рендерятся и проходят локальный Tesseract rus+eng. При отсутствии OCR или недостаточном тексте загрузка завершается ошибкой, страница не пропускается молча. OCR таблиц/чертежей ограничен распознаваемыми подписями и текстом; пространственные размеры или визуальное устройство не выдумываются. Для инженерных выводов по геометрии потребуется локальная multimodal модель и отдельный набор оценок.

Sanitizer скрывает email/телефоны, подозрительные инструкции блокируют ingestion. Документ, версия и hash записываются транзакционно. Page chunks около 1000 символов, overlap 100; section/token parser вынесен как известное ограничение. Обнаруженные asset IDs создают Entity. В пределах чанка создаётся CO_MENTION со ссылкой на source chunk. Онтология: Document, Chunk, Entity; роли/сессии - метаданные SQL. CO_MENTION показывает совместное упоминание, а не автоматически извлеченную причинную или физическую связь. Точное утверждение остаётся в цитате источника.

Канонические ACL/чанки/edges живут в SQLite; производные Qdrant и Neo4j синхронизируются отдельным профилем. Qdrant содержит vector и ACL payload без текста; Neo4j - Document/Chunk/Entity и source edges без содержимого/бюджетов. Neo4j browser имеет отдельную административную аутентификацию. Пользователь видит только /graph, который применяет RBAC. DB ports не публикуются в DMZ.

Поиск и reasoning

Guard проверяет ввод → Planner проверяет session ownership/память → Retriever ищет только среди разрешённых чанков → Graph Tool проходит provenance edges до двух hops → Rerank ограничивает контекст → Generator выбирает цитаты → Output Validator повторно проверяет ACL и точное вхождение цитат. При отсутствии evidence первый проход возвращается к Planner для попытки использовать разрешённую память на указательное местоимение. Второй miss завершает no_evidence. Это state machine с условными переходами и bounded loop, не линейная цепочка.

Для минимального offline corpus используется lexical hashing vector и pair overlap reranking. Это запускаемая проверка графа/ACL, а не заявленный multilingual semantic benchmark. Профиль pretrained embeddings подключает локальный SentenceTransformer через EMBEDDING_MODEL_DIR с

local_files_only. Смена embedding model/dimension требует rebuild collection. Cross-encoder reranker остаётся следующим обоснованным расширением для неоднозначного свободного текста. MVP возвращает точные подтверждённые цитаты; свободное резюме без проверки фактов не выпускается.

LLM Serving GPU profile использует vLLM с готовой Qwen2.5-7B-Instruct-AWQ, FP16 KV, prefix caching, context cap 4096, max-num-seqs=4 и GPU memory utilization=0,80, kernel AWQ Marlin. 7B INT4 weights порядка 3,5 GB до overhead; фактический artifact больше. Для Qwen2.5 28 layers×4 KV-heads×128 head_dim×2 K/V×2 bytes даёт 57 344 bytes/token, максимум 4×4096 - около 0,875 GiB KV. Это расчёт резерва на RTX4070 12 GiB. GPU-профиль реально запущен: лог vLLM подтверждает загрузку weights, KV и inference. Размер весов в generic AWQ runtime 5,2034 GiB; дисковые safetensors занимают 5,57 GB. Точные резервы финального Marlin-профиля сохранены в журнале; использование всей GPU включает другие приложения рабочего стола. CPU offload может снизить VRAM потребность ценой latency, применяется только после benchmark.

OpenAI-compatible /v1/chat/completions - формат HTTP локального сервера, не обращение к компании OpenAI. Client допускает только явный INTERNAL_HOSTS allowlist. Администратор не задаёт внешние URL через пользовательский запрос. В extractive mode LLM полностью отсутствует, usage tokens=0; режим указан в каждом ответе.

Security by Design

Identity поступает из Bearer token серверного provisioning-файла. Роли/tenant не принимаются из JSON запроса. В prod этот механизм заменяет OIDC/корпоративный SSO на gateway, без изменения Authorizer. Запросы SQL параметризованы, пользовательские tool URLs/SQL/shell запрещены. Input regex и PII regex служат первым слоем, не считаются полной защитой от jailbreak/NER. Непрямые инъекции тестируются отдельно, возможности модели ограничены read-only retrieval.

RBAC инвариант: любой возвращённый quote/graph edge имеет разрешённый tenant и grant source document в момент финальной проверки. Shared Entity не наследует совокупность прав документов: edge разрешён только по своему source chunk. Выборка сначала фильтруется по ACL, потом ранжируется. После отзыва grant история очищается логически на чтении, индекс может быть stale, но каноническая reauthorization блокирует выдачу. Cache свободного ответа не реализован, поэтому межпользовательского answer cache нет.

Deployment: DMZ содержит TLS reverse proxy и browser channel, Internal Control - agents/ingestion, Internal Data - DBs/GPU. Между зонами allowlist ports, mTLS, service identities, default-deny egress. Vault выдаёт short-lived DB credentials и хранит секреты вне state. Эти production controls задокументированы; локальный Compose использует internal network без egress для backend/DB/GPU, Nginx gateway с отдельной edge-сетью, loopback ports, generated secrets и non-root/read-only backend. Gateway имеет фиксированные upstream: пользователь не может превратить его в произвольный proxy. Фактический тест backend egress выполнен. Развёрнутый MVP не объявляется сертифицированным или полностью hardened.

В целевой Deployment L7 reverse proxy балансирует запросы между готовыми CPU-репликами API, а внутренний serving router направляет inference в прогретые GPU-реплики с учётом длины очереди и readiness. Несколько API-реплик требуют общей PostgreSQL для ACL/истории и общего checkpointer (ADR-P04); отдельные SQLite на каждой реплике недопустимы. Локальный MVP использует одну API-реплику и одну GPU, поэтому эта балансировка является проектным расширением.

Observability и эксплуатация

OpenTelemetry spans query, agent.guard/plan/retrieve/expand/rerank/generate/validate, llm.generate отправляются OTLP в Jaeger. Нет question/answer/source text в span attributes. Prometheus: requests/status, latency histogram, actual input/output tokens. vLLM публикует отдельные метрики serving. Grafana provisioned datasource и dashboard. Логи содержат request ID/status/evidence_count. История и audit содержат минимальные псевдонимы, не секреты.

Readiness для GPU требует фактической загрузки весов и пробного inference; liveness backend /health отражает API, не готовность всех зависимостей. Нельзя использовать этот health как production readiness. Backup SQL через consistent snapshot, сохранение source hash/version и ACL, восстановление derived indexes из canonical store. Предлагаемые RPO 24 ч/RTO 4 ч подтверждаются restore exercise, а не наличием volume.

Проверка и пределы MVP

backend.check проверяет двухшаговую связь, RBAC/tenant, shared entity bridge, session ownership, revoke во время generation, PII, poisoned docs, внешний inference URL, invented quote и HTTP/SSE. benchmark.py измеряет in-process ASGI extractive workload и пишет scope рядом с числами. Контейнерная интеграция и текущий статус GPU/video указаны в проверки.md.

Решение подготавливается к сдаче минимум за восемь дней до защиты. Дата защиты отсутствует, календарный дедлайн не выдуман. Публичный Git remote и публикация реальных данных не требуется для локального monorepo; копия проекта подготовлена для переноса в разрешённый репозиторий.